Let's demonstrate this with an example. When we use a command to create a filename appending the string _file for the variables.

```
[me@linuxbox ~]$ a="foo"
[me@linuxbox ~]$ echo "$a _ file"
```

It gives nothing as the shell looks to expand a variable named a_file rather than appending a. To solve this, adding braces around the variable.

```
[me@linuxbox ~]$ echo "${a} _ file"
foo _ file
```

Similarly, when accessing the positional parameters greater than they are surrounded by braces to get the desired result. (For instance ${12})

Expansions To Manage Empty Variables

There are several rules to handle empty or non-existent variables. These expansions are applied when assigning default values to parameters and handling missing positional parameters. `${parameter:-word}`

So when a parameter is unset or is empty, it takes the value of the word, while it takes the value of a parameter in case it is not empty.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:-"substitute value if unset"}
substitute value if unset
[me@linuxbox ~]$ echo $foo
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:-"substitute value if unset"}
bar
[me@linuxbox ~]$ echo $foo
bar
${parameter:=word}
```

Again, if the parameter is unset or empty, the expansion results with the value of the word while it takes the value of the parameter if it is not empty.

```
[me@linuxbox ~]$ foo=
[me@linuxbox ~]$ echo ${foo:="default value if unset"}
default value if unset
[me@linuxbox ~]$ echo $foo
default value if unset
[me@linuxbox ~]$ foo=bar
[me@linuxbox ~]$ echo ${foo:="default value if unset"}
[me@linuxbox ~]$ echo $foo
bar
```